

Contents

1	Cyclic Codes	2
1.1	Historical Development of Cyclic Codes	2
1.1.1	Shift in Research Perspective	3
1.2	Cyclic Codes	4
1.2.1	Example 1.1: Checking a Cyclic Shift	4
1.2.2	Example 1.2: Deciding Whether a Code Is Cyclic	4
1.2.3	Example 1.3: A Linear Code That Is Not Cyclic	5
1.3	Cyclic-Shift Invariance and Implementation	5
1.3.1	The Drawback of General Linear Codes	5
1.3.2	The Cyclic-Code Alternative: Shift Registers	5
1.3.3	Example: Matrix Multiplication vs. LFSR for a $[7, 4]_2$ Code	5
1.3.4	Why the Encoder Requires Seven Clock Cycles (Non-Systematic Encoding)	6
1.3.5	Why Does “Cyclic” Enable Shift Registers?	8
1.4	Polynomial Representation of Cyclic Codes	9
1.4.1	Example 2.1: Vector $\leftrightarrow$ Polynomial Conversion	9
1.4.2	Example 2.2: Cyclic Shift as a Polynomial Operation	9
1.4.3	Example 2.3: When the Highest-Degree Term Wraps Around	9
1.5	Generator Polynomials and Their Properties	10
1.5.1	Example 3.1: Factorization of $X^7 - 1$ and Possible Generator Polynomials	10
1.5.2	Example 3.2: Non-Systematic Encoding for a $[7, 4]_2$ Code	10
1.5.3	Example 3.3: Another Non-Systematic Encoding Example	11
1.6	Parity-Check Polynomials	11
1.6.1	Example 4.1: Parity-Check Polynomial for a $[7, 4]_2$ Code	11
1.6.2	Example 4.2: Validating a Codeword	11
1.7	Generator Matrices	11
1.7.1	Example 5.1: Generator Matrix for a $[7, 4]_2$ Cyclic Code	12
1.7.2	Example 5.2: Encoding with the Generator Matrix	12
1.7.3	Example 5.3: A $(7, 3)$ Cyclic Code	12
1.8	Parity-Check Matrices	13
1.8.1	Example 6.1: Parity-Check Matrix for a $[7, 4]_2$ Cyclic Code	13
1.8.2	Example 6.2: Verifying Orthogonality	13
1.8.3	Example 6.3: Error Detection with the Parity-Check Matrix	14
1.9	Systematic and Non-Systematic Encoding	14
1.9.1	Example 7.1: Two Encodings of the Same Message	14
1.10	Practical Systematic Encoding	15
1.10.1	Example 8.1: Systematic Encoding of $(1, 0, 1, 1)$	15
1.10.2	Example 8.2: Systematic Encoding of $(1, 1, 0, 0)$	16
1.10.3	Example 8.3: Systematic Encoding of $(0, 0, 0, 1)$	16
1.10.4	Example 8.4: Full Table of Systematic Codewords for the $[7, 4]_2$ Code	16
1.11	Syndromes and Error Detection	17
1.11.1	Example 9.1: Error-Free Reception	17
1.11.2	Example 9.2: A Single-Bit Error at Position X^2	18
1.11.3	Example 9.3: Complete Syndrome Table for Single-Bit Errors in the $[7, 4]_2$ Code	18
1.11.4	Example 9.4: Error Correction Using the Syndrome	18
1.11.5	Example 9.5: The Case of a Two-Bit Error	19
1.12	Decoding of Cyclic Codes	19
1.12.1	Step 1: Syndrome Calculation	19
1.12.2	Step 2: Error-Pattern Search	19
1.12.3	Step 3: Error Correction	19
1.12.4	Step 4: Message Extraction	20
1.13	How Is the Error-Correcting Capability of a Cyclic Code Determined?	20
1.13.1	t -Error-Correcting Capability	20

1.13.2	Example 11.1: Error-Correcting Capability of Representative Cyclic Codes	20
1.13.3	Limitation of a General Cyclic Code	20
1.13.4	BCH Bound: Guaranteed Designed Distance	20
1.13.5	Example 11.2: Applying the BCH Bound	21
1.13.6	Example 11.3: General Cyclic Code vs. BCH Code	21
1.14	Algebraic Structure of Cyclic Codes	21
1.14.1	Proof	21
1.14.2	Example 10.1: Checking the Ideal Structure	22
1.14.3	Example 10.2: Why Must $g(X)$ Divide $X^n - 1$?	22
1.15	Number of Ideals in the Quotient Ring and Determination of the Generator Polynomial . . .	22
1.15.1	Main Theorem: Counting Ideals	22
1.15.2	Procedure for Determining Generator Polynomials	23
1.15.3	Example 13.1: Listing All Ideals for $n = 7$ over $\mathbb{F}_2$	24
1.15.4	Example 13.2: Number of Ideals for $n = 15$ over $\mathbb{F}_2$	24
1.15.5	Example 13.3: Ideal Lattice and Inclusion Relations	24
1.15.6	A Caution Concerning $\gcd(n, q) \neq 1$	25
1.16	Why Are Cyclic Codes Important in Practice?	25
1.16.1	The Principle of the LFSR Structure	25
1.16.2	Advantage of Real-Time Processing	25
1.16.3	CRC in Network Transmission	26
1.16.4	Example 11.1: LFSR Encoder for $g(X) = 1 + X + X^3$	26
1.17	Representative Families of Cyclic Codes	26
1.17.1	Example 15.1: CRC-8 Generator Polynomial	27
1.17.2	Example 15.2: CRC-32 Used in Ethernet	27
1.18	What Are Quasi-Cyclic Codes and the Post-Quantum Cryptosystem HQC?	27
1.18.1	Quasi-Cyclic Codes (QC)	27
1.18.2	Use in HQC (Hamming Quasi-Cyclic)	27
1.18.3	Example 16.1: Structure of a Circulant Block	27
1.19	Exercises	28
1.19.1	Exercise 1. Determining Whether a Code Is Cyclic	28
1.19.2	Exercise 2. Finding Generator Polynomials	28
1.19.3	Exercise 3. Practice with Systematic Encoding	28
1.19.4	Exercise 4. Syndrome Computation and Error Correction	28
1.19.5	Exercise 5. Generator and Parity-Check Matrices	29
1.19.6	Exercise 6. Understanding the Algebraic Structure	29
1.19.7	Exercise 7. Comprehensive Problem	29
1.20	SageMath Code	29
1.21	References	29

1 Cyclic Codes

This note presents an introduction to cyclic codes in a style intended to be closer to a mathematical monograph than to informal lecture notes. The exposition emphasizes precise definitions, structural results, representative examples, and computational practice.

1.1 Historical Development of Cyclic Codes

We begin by recording several principal milestones in the historical development of cyclic codes.

Year	Milestone	Description
1948	Foundation of Coding Theory	Shannon established information theory and proved the existence of reliable codes for noisy channels [1].
1950	Hamming Codes	Hamming introduced the first systematic linear codes capable of correcting a single error [2].
1954	Reed–Muller Codes	Reed and Muller introduced a family of multiple-error-correcting codes arising from Boolean functions [3][4].
1957	Algebraic Formulation of Cyclic Codes	Prange showed that cyclic codes may be identified with ideals in the quotient ring $\mathbb{F}_q[X]/\langle X^n - 1 \rangle$ [5].
1959–1960	BCH Codes	Hocquenghem, Bose, and Ray-Chaudhuri introduced BCH codes, providing a systematic algebraic construction with prescribed designed distance [6][7].
1960	Reed–Solomon Codes	Reed and Solomon introduced an important class of nonbinary cyclic codes with strong symbol-level correction capability [8].
1961	Algebraic Decoding	Peterson developed an algebraic decoding method for BCH codes and published a foundational text on error-correcting codes [9].
1968	Berlekamp–Massey Algorithm	Berlekamp and Massey introduced an efficient iterative procedure for determining the error-locator polynomial [10][11].
1970s	Standardization of CRC	Cyclic redundancy check became a standard method of error detection because cyclic codes admit efficient LFSR-based implementation.
1977	Goppa Codes and Code-Based Cryptography	Goppa introduced algebraic geometry codes, and McEliece employed Goppa codes in a fundamental public-key cryptosystem [12][13].
Since the 1990s	Modern Developments	Despite the emergence of turbo codes and LDPC codes, cyclic codes have remained important because of their algebraic transparency, implementation efficiency, and cryptographic relevance [14][15].

1.1.1 Shift in Research Perspective

A noteworthy feature of the subject is that its dominant point of view has changed over time.

Early stage (1950s): engineering needs -> discovery of algebraic structure

"If we build codes that remain valid under cyclic shifts, they can be implemented with LFSRs"
-> search for linear codes closed under cyclic shift (Prange, 1957)
-> mathematical analysis reveals equivalence with ideals of $\mathbb{F}_q[X]/\langle X^n - 1 \rangle$
-> existing algebra (ideal theory) becomes immediately applicable

In this period, the desired engineering property came first, and the ideal structure was discovered in the process of formalizing it.

Transition period (from 1959): algebraic structure -> code design with target performance

"If we prescribe roots of the generator polynomial over an extension field, we can guarantee a minimum δ "
-> BCH codes (Hocquenghem 1959, Bose-Ray-Chaudhuri 1960)
-> Reed-Solomon codes (Reed-Solomon 1960)
-> the designed distance δ can be chosen in advance to control t -error-correction capability

From this point on, algebraic structure was actively exploited to design codes with desired error-correcting performance.

Modern stage (since the 1970s): bidirectional interaction

algebraic structure $\leftrightarrow$ engineering applications

- > CRC: exploit cyclic structure for hardware-efficient error detection
- > Goppa/McEliece: coding theory -> public-key cryptography
- > HQC: quasi-cyclic structure -> smaller keys in post-quantum cryptography

Today, algebraic structure and engineering applications continue to influence one another in both directions.

1.2 Cyclic Codes

A cyclic code is a special subclass of linear block codes that satisfies two key properties simultaneously: **linearity** and **invariance under cyclic shift**.

A linear code $\mathcal{C}$ of length n is called a cyclic code if it satisfies both of the following:

1. **Linearity**: for any two codewords $\mathbf{c}_1, \mathbf{c}_2 \in \mathcal{C}$, we have $\mathbf{c}_1 + \mathbf{c}_2 \in \mathcal{C}$.
2. **Cyclic shift property**: if $\mathbf{c} = (c_0, c_1, \dots, c_{n-1}) \in \mathcal{C}$, then the vector obtained by a one-step cyclic right shift,

$$\mathbf{c} \ggg^1 = (c_{n-1}, c_0, c_1, \dots, c_{n-2}),$$

must also belong to $\mathcal{C}$.

Therefore any cyclic shift by two steps, three steps, or in fact any number of steps also yields a valid codeword.

1.2.1 Example 1.1: Checking a Cyclic Shift

In a binary $[7, 4]_2$ cyclic code, suppose

$$\mathbf{c} = (1, 0, 0, 1, 0, 1, 1)$$

is a codeword. Then:

- 1-step shift: $\mathbf{c} \ggg^1 = (1, 1, 0, 0, 1, 0, 1) \rightarrow$ codeword
- 2-step shift: $\mathbf{c} \ggg^2 = (1, 1, 1, 0, 0, 1, 0) \rightarrow$ codeword
- 3-step shift: $\mathbf{c} \ggg^3 = (0, 1, 1, 1, 0, 0, 1) \rightarrow$ codeword

All of these vectors must belong to the code space $\mathcal{C}$.

1.2.2 Example 1.2: Deciding Whether a Code Is Cyclic

Let $n = 4$ and consider the set

$$\mathcal{C} = \{(0, 0, 0, 0), (1, 0, 1, 0), (0, 1, 0, 1), (1, 1, 1, 1)\}.$$

- Linearity check: $(1, 0, 1, 0) \oplus (0, 1, 0, 1) = (1, 1, 1, 1) \in \mathcal{C}$.
- Cyclic-shift check:
 - $(1, 0, 1, 0)$ shifted once gives $(0, 1, 0, 1) \in \mathcal{C}$.
 - $(0, 1, 0, 1)$ shifted once gives $(1, 0, 1, 0) \in \mathcal{C}$.
 - $(1, 1, 1, 1)$ shifted once gives $(1, 1, 1, 1) \in \mathcal{C}$.

Hence this is a cyclic code.

1.2.3 Example 1.3: A Linear Code That Is Not Cyclic

Let

$$\mathcal{C}' = \{(0, 0, 0, 0), (1, 1, 0, 0), (0, 0, 1, 1), (1, 1, 1, 1)\}.$$

Shifting $(1, 1, 0, 0)$ one step cyclically gives $(0, 1, 1, 0)$, which is not in $\mathcal{C}'$. Therefore $\mathcal{C}'$ is linear, but **not** cyclic.

1.3 Cyclic-Shift Invariance and Implementation

Cyclic codes were originally studied not because of abstract algebra, but because of **hardware efficiency**. Prange's key observation in the 1950s was that if a code is closed under cyclic shift, then encoding and error detection can be implemented using nothing more than shift registers.

1.3.1 The Drawback of General Linear Codes

For a general linear $[n, k]_q$ code, encoding is done by matrix multiplication:

$$\mathbf{c} = \mathbf{m} \cdot G.$$

This means:

- the $k \times n$ generator matrix G must be stored in memory,
- $k \cdot n$ additions and multiplications are needed,
- computation cannot begin until the whole message block has arrived.

For example, for a $[31, 21]_2$ code, one must store $21 \times 31 = 651$ matrix entries and perform 651 binary operations for every block.

1.3.2 The Cyclic-Code Alternative: Shift Registers

For cyclic codes, every codeword is a multiple of the generator polynomial $g(X)$, and encoding reduces to polynomial multiplication or polynomial division with remainder. These operations can be implemented using an LFSR (linear feedback shift register):

- only $r = n - k$ one-bit registers are required,
- the input stream is processed one bit per clock,
- after the last bit passes through the circuit, the register contents are exactly the parity bits or syndrome.

For a $[31, 21]_2$ code, 10 registers plus a few XOR gates are enough.

1.3.3 Example: Matrix Multiplication vs. LFSR for a $[7, 4]_2$ Code

Item	General linear code (matrix multiplication)	Cyclic code (LFSR)
Storage	$G \in GF(2)^{4 \times 7} \rightarrow 28$ bits	$g(X) = 1 + X + X^3 \rightarrow 4$ bits (coefficients $[1, 1, 0, 1]$)
Operations per block	$4 \times 7 = 28$ AND/XOR operations	7 clocks (roughly one XOR per bit)
Processing mode	Batch processing after full block arrival	Real-time, bit-by-bit
Latency	Proportional to block length	Almost zero
Circuit complexity	n XOR trees driven by k inputs	r flip-flops plus XOR gates

Non-systematic encoding $c(X) = m(X)g(X)$ is, mathematically, a convolution of two polynomials. For a $[7, 4]_2$ code the message has 4 bits, the generator polynomial has degree 3, and the codeword has length 7.

$$m(X) = m_0 + m_1 X + m_2 X^2 + m_3 X^3,$$

$$q(X) = q_0 + q_1X + q_2X^2 + q_3X^3.$$

$$c(X) = \sum_{k=0}^6 c_k X^k,$$

- $c_0 = m_0 g_0$
- $c_1 = m_0 g_1 + m_1 g_0$
- $c_2 = m_0 g_2 + m_1 g_1 + m_2 g_0$
- $c_3 = m_0 g_3 + m_1 g_2 + m_2 g_1 + m_3 g_0$
- $c_4 = m_1 g_3 + m_2 g_2 + m_3 g_1$
- $c_5 = m_2 g_3 + m_3 g_2$
- $c_6 = m_3 g_3$

Clock	Input	Output	Explanation
1	m_0	c_0	The first codeword bit appears immediately
2	m_1	c_1	Contributions of m_0 and m_1 combine
3	m_2	c_2	Message input finishes, but the tail has not flushed out yet
4	m_3	c_3	
5	0	c_4	Feed zeros to flush the remaining state
6	0	c_5	Final output bit
7	0	c_6	

Message input m_k

+

(x) $g_0=1$ (x) $g_1=1$ (x) $g_2=0$ (x) $g_3=1$

$\begin{array}{cccc} | & & | & & | & & | \\ v & & v & & v & & v \end{array}$

0 --> (+)----[D0]----(+)--[D1]----(+)--[D2]----(+)--> Codeword output c_k

How it works: 1. **Input phase (clocks 1-4):** message bits m_0, m_1, m_2, m_3 are fed in sequence. At each clock the current input bit is multiplied by each coefficient of $g(X)$ and combined with delayed values stored in the registers. 2. **Flush phase (clocks 5-7):** after the message bits are exhausted, zeros are shifted in. The intermediate values stored in D_0, D_1, D_2 are flushed out, yielding c_4, c_5, c_6 . 3. **Result:** after 7 clocks all registers are cleared and the 7-bit codeword has been fully emitted.

Hence the total number of clocks is

$$4 + 3 = 7,$$

which equals the codeword length n .

4. Step-by-Step Visualization Let the message be $\mathbf{m} = (1, 1, 0, 1)$ and let $g(X) = 1 + X + X^3$.

Initial state: all registers are zero.

Input m_k: -

Registers: [D0:0] [D1:0] [D2:0]

Output c_k: -

Clock 1: input $m_0 = 1$

- $c_0 = 1 \cdot 1 \oplus 0 = 1$
- New state: $D_0 \leftarrow 1, D_1 \leftarrow 0, D_2 \leftarrow 1$

Input m_k: 1

$\begin{array}{cccc} | & & & & \\ v & (x1) & (x1) & (x0) & (x1) \\ | & | & | & | & | \end{array}$

0 --(+)--[D0]--(+)--[D1]--(+)--[D2]--(+)--> Output c_0: 1

$\begin{array}{ccc} \text{(New:1)} & \text{(New:0)} & \text{(New:1)} \end{array}$

Clock 2: input $m_1 = 1$

- $c_1 = 1 \cdot 1 \oplus D_0 = 1 \oplus 1 = 0$
- New state: $D_0 \leftarrow 1, D_1 \leftarrow 1, D_2 \leftarrow 1$

Input m_k: 1

$\begin{array}{c} | \\ v \end{array}$

0 --(+)--[D0:1]--(+)--[D1:0]--(+)--[D2:1]--(+)--> Output c_1: 0

$\begin{array}{ccc} \text{(New:1)} & \text{(New:1)} & \text{(New:1)} \end{array}$

Clock 3: input $m_2 = 0$

- $c_2 = 0 \cdot 1 \oplus D_0 = 1$
- New state: $D_0 \leftarrow 1, D_1 \leftarrow 1, D_2 \leftarrow 0$

Input m_k: 0

$\begin{array}{c} | \\ v \end{array}$

0 --(+)--[D0:1]--(+)--[D1:1]--(+)--[D2:1]--(+)--> Output c_2: 1

$\begin{array}{ccc} \text{(New:1)} & \text{(New:1)} & \text{(New:0)} \end{array}$

Clock 4: input $m_3 = 1$

- $c_3 = 1 \cdot 1 \oplus D_0 = 0$
- New state: $D_0 \leftarrow 0, D_1 \leftarrow 0, D_2 \leftarrow 1$

Input m_k: 1
 |
 v
 0 --(+)--[D0:1]--(+)--[D1:1]--(+)--[D2:0]--(+)--> Output c_3: 0
 (New:0) (New:0) (New:1)

Clock 5: flush with input 0

- $c_4 = 0 \cdot g_0 \oplus D_0 = 0$
- New state: $D_0 \leftarrow 0, D_1 \leftarrow 1, D_2 \leftarrow 0$

Input m_k: 0
 |
 v
 0 --(+)--[D0:0]--(+)--[D1:0]--(+)--[D2:1]--(+)--> Output c_4: 0
 (New:0) (New:1) (New:0)

Clock 6: flush again

- $c_5 = 0 \cdot g_0 \oplus D_0 = 0$
- New state: $D_0 \leftarrow 1, D_1 \leftarrow 0, D_2 \leftarrow 0$

Input m_k: 0
 |
 v
 0 --(+)--[D0:0]--(+)--[D1:1]--(+)--[D2:0]--(+)--> Output c_5: 0
 (New:1) (New:0) (New:0)

Clock 7: final output

- $c_6 = 0 \cdot g_0 \oplus D_0 = 1$
- All registers return to zero.

Input m_k: 0
 |
 v
 0 --(+)--[D0:1]--(+)--[D1:0]--(+)--[D2:0]--(+)--> Output c_6: 1
 (New:0) (New:0) (New:0)

Final result:

$$\mathbf{c} = (1, 0, 1, 0, 0, 0, 1).$$

This matches the polynomial product

$$c(X) = (1 + X + X^3)(1 + X + X^3) = 1 + X^2 + X^6.$$

1.3.5 Why Does “Cyclic” Enable Shift Registers?

The key correspondence is

$$\text{cyclic shift} \longleftrightarrow X \cdot c(X) \pmod{X^n - 1}.$$

The basic action of a shift register - moving all bits by one position and injecting feedback - is exactly multiplication by X followed by reduction modulo $X^n - 1$. This is why cyclic codes admit LFSR implementations, whereas general linear codes do not.

In summary:

cyclic-shift invariance

- > codewords are multiples of $g(X)$
- > encoding becomes polynomial division / multiplication
- > implementable by LFSR
- > storage $O(n-k)$, computation $O(n)$, real-time processing

This is the fundamental reason cyclic codes became so important, and why CRC remains the standard error-detection mechanism in protocols such as Ethernet and USB.

1.4 Polynomial Representation of Cyclic Codes

A vector of length n ,

$$\mathbf{c} = (c_0, c_1, \dots, c_{n-1}),$$

can be mapped to a polynomial of degree at most $n - 1$:

$$c(X) = c_0 + c_1X + c_2X^2 + \dots + c_{n-1}X^{n-1}.$$

Under this representation, a one-step cyclic right shift corresponds exactly to

$$X \cdot c(X) \pmod{X^n - 1}.$$

Therefore all algebraic operations on a cyclic code naturally live in the quotient ring

$$R_n = \mathbb{F}_q[X]/\langle X^n - 1 \rangle.$$

1.4.1 Example 2.1: Vector $\leftrightarrow$ Polynomial Conversion

For $n = 7$:

Vector	Polynomial
(1, 0, 1, 1, 0, 0, 0)	$1 + X^2 + X^3$
(0, 1, 0, 0, 1, 1, 0)	$X + X^4 + X^5$
(1, 1, 1, 1, 1, 1, 1)	$1 + X + X^2 + X^3 + X^4 + X^5 + X^6$

1.4.2 Example 2.2: Cyclic Shift as a Polynomial Operation

Let $n = 7$ and $c(X) = 1 + X^2 + X^3$. Then

$$Xc(X) = X + X^3 + X^4.$$

Since the degree is still below 7, reduction modulo $X^7 - 1$ changes nothing. The corresponding vector is (0, 1, 0, 1, 1, 0, 0), which is exactly the one-step cyclic shift of (1, 0, 1, 1, 0, 0, 0).

1.4.3 Example 2.3: When the Highest-Degree Term Wraps Around

Let $n = 7$ and $c(X) = X^4 + X^5 + X^6$. Then

$$Xc(X) = X^5 + X^6 + X^7.$$

Since $X^7 \equiv 1 \pmod{X^7 - 1}$,

$$Xc(X) \pmod{X^7 - 1} = 1 + X^5 + X^6.$$

Thus

$$(0, 0, 0, 0, 1, 1, 1) \xrightarrow{\text{1-step shift}} (1, 0, 0, 0, 0, 1, 1).$$

1.5 Generator Polynomials and Their Properties

Every cyclic code is completely determined by a unique monic polynomial $g(X)$ of **smallest degree** in the code. This polynomial is called the **generator polynomial**.

Key properties:

- $g(X)$ must divide $X^n - 1$:

$$X^n - 1 = g(X)h(X).$$

- If $\deg g = r = n - k$, then the code dimension is k .
- Every codeword is a multiple of $g(X)$:

$$c(X) = m(X)g(X).$$

For **non-systematic encoding**, one simply multiplies the message polynomial by the generator polynomial:

$$c(X) = m(X)g(X).$$

1.5.1 Example 3.1: Factorization of $X^7 - 1$ and Possible Generator Polynomials

Over $GF(2)$, we have $-1 = 1$, so $X^7 - 1 = X^7 + 1$, and it factors as

$$X^7 + 1 = (1 + X)(1 + X + X^3)(1 + X^2 + X^3).$$

Any divisor of this polynomial can serve as a generator polynomial:

$g(X)$	$\deg(g) = n - k$	$[n, k]_2$	Number of codewords 2^k
$1 + X$	1	$[7, 6]_2$	64
$1 + X + X^3$	3	$[7, 4]_2$	16
$1 + X^2 + X^3$	3	$[7, 4]_2$	16
$(1 + X)(1 + X + X^3) =$ $1 + X^2 + X^3 + X^4$	4	$[7, 3]_2$	8
$(1 + X)(1 + X^2 + X^3) =$ $1 + X + X^2 + X^4$	4	$[7, 3]_2$	8
$(1 + X + X^3)(1 + X^2 + X^3) =$ $1 + X + X^2 + X^3 + X^4 + X^5 + X^6$	6	$[7, 1]_2$	2
$X^7 + 1$	7	$[7, 0]_2$	1 (trivial)

1.5.2 Example 3.2: Non-Systematic Encoding for a $[7, 4]_2$ Code

Let $g(X) = 1 + X + X^3$ and let the message be

$$\mathbf{m} = (1, 1, 0, 1), \quad m(X) = 1 + X + X^3.$$

Then

$$\begin{aligned}
 c(X) &= m(X)g(X) \\
 &= (1 + X + X^3)(1 + X + X^3) \\
 &= 1 + X + X^3 + X + X^2 + X^4 + X^3 + X^4 + X^6 \\
 &= 1 + (1 + 1)X + X^2 + (1 + 1)X^3 + (1 + 1)X^4 + X^6 \\
 &= 1 + X^2 + X^6.
 \end{aligned}$$

Thus the codeword vector is

$$\mathbf{c} = (1, 0, 1, 0, 0, 0, 1).$$

1.5.3 Example 3.3: Another Non-Systematic Encoding Example

With the same generator polynomial, let the message be

$$\mathbf{m} = (0, 1, 0, 0), \quad m(X) = X.$$

Then

$$c(X) = X(1 + X + X^3) = X + X^2 + X^4.$$

So the codeword is

$$\mathbf{c} = (0, 1, 1, 0, 1, 0, 0).$$

This is exactly the coefficient vector of $Xg(X)$, i.e. the second row of the generator matrix.

1.6 Parity-Check Polynomials

The quotient obtained by dividing $X^n - 1$ by the generator polynomial is called the **parity-check polynomial**:

$$h(X) = \frac{X^n - 1}{g(X)}.$$

Its degree is k . To check whether a polynomial $v(X)$ is a valid codeword, one tests

$$v(X)h(X) \equiv 0 \pmod{X^n - 1}.$$

1.6.1 Example 4.1: Parity-Check Polynomial for a $[7, 4]_2$ Code

If $g(X) = 1 + X + X^3$, then

$$h(X) = \frac{X^7 + 1}{1 + X + X^3} = 1 + X + X^2 + X^4.$$

Verification:

$$g(X)h(X) = (1 + X + X^3)(1 + X + X^2 + X^4) = X^7 + 1.$$

Indeed, term-by-term cancellation in $GF(2)$ leaves only $1 + X^7$.

1.6.2 Example 4.2: Validating a Codeword

Consider the codeword

$$c(X) = X + X^2 + X^4.$$

Then

$$c(X)h(X) = (X + X^2 + X^4)(1 + X + X^2 + X^4).$$

Expanding and reducing modulo $X^7 + 1$ yields 0, so $c(X)$ is a valid codeword.

1.7 Generator Matrices

Let

$$g(X) = g_0 + g_1X + \cdots + g_{n-k}X^{n-k}.$$

Then the generator matrix G is built by taking the coefficient vector of $g(X)$ and shifting it one position to the right in each row:

$$G = \begin{bmatrix} g_0 & g_1 & g_2 & \cdots & g_{n-k} & 0 & \cdots & 0 \\ 0 & g_0 & g_1 & \cdots & g_{n-k-1} & g_{n-k} & \cdots & 0 \\ \vdots & & & \ddots & & & \ddots & \vdots \\ 0 & \cdots & 0 & g_0 & g_1 & \cdots & \cdots & g_{n-k} \end{bmatrix}.$$

The i -th row corresponds to the coefficient vector of $X^i g(X)$.

Why is this a valid generator matrix? 1. $g(X)$ is a codeword, and by cyclic-shift invariance so are $Xg(X), X^2g(X), \dots, X^{k-1}g(X)$. 2. These vectors are linearly independent because their highest nonzero terms occur in different positions. 3. A k -dimensional code containing k linearly independent codewords has found a basis.

1.7.1 Example 5.1: Generator Matrix for a $[7, 4]_2$ Cyclic Code

Let $g(X) = 1 + X + X^3$, whose coefficient vector is $(1, 1, 0, 1)$. Then

$$G = \begin{bmatrix} 1 & 1 & 0 & 1 & 0 & 0 & 0 \\ 0 & 1 & 1 & 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & 1 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 & 1 & 0 & 1 \end{bmatrix}.$$

Interpretation of the rows:

- Row 0: $g(X) = 1 + X + X^3$
- Row 1: $Xg(X) = X + X^2 + X^4$
- Row 2: $X^2g(X) = X^2 + X^3 + X^5$
- Row 3: $X^3g(X) = X^3 + X^4 + X^6$

1.7.2 Example 5.2: Encoding with the Generator Matrix

Let $\mathbf{m} = (1, 0, 1, 1)$. Then

$$\mathbf{c} = \mathbf{m}G = (1, 0, 1, 1) \begin{bmatrix} 1 & 1 & 0 & 1 & 0 & 0 & 0 \\ 0 & 1 & 1 & 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & 1 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 & 1 & 0 & 1 \end{bmatrix}.$$

Carrying out the multiplication over $GF(2)$ gives

$$\mathbf{c} = (1, 1, 1, 1, 1, 1, 1).$$

Thus

$$c(X) = 1 + X + X^2 + X^3 + X^4 + X^5 + X^6,$$

and this is divisible by $g(X)$.

1.7.3 Example 5.3: A $(7, 3)$ Cyclic Code

Let $g(X) = 1 + X^2 + X^3 + X^4$. Then $k = 3$ and

$$G = \begin{bmatrix} 1 & 0 & 1 & 1 & 1 & 0 & 0 \\ 0 & 1 & 0 & 1 & 1 & 1 & 0 \\ 0 & 0 & 1 & 0 & 1 & 1 & 1 \end{bmatrix}.$$

For message $\mathbf{m} = (1, 1, 0)$,

$$\mathbf{c} = (1, 1, 0)G = (1, 1, 1, 0, 0, 1, 0).$$

1.8 Parity-Check Matrices

Let

$$h(X) = h_0 + h_1X + \cdots + h_kX^k.$$

To construct the parity-check matrix H , reverse the coefficient order of $h(X)$ and shift that reversed vector across the rows:

$$H = \begin{bmatrix} h_k & h_{k-1} & \cdots & h_0 & 0 & \cdots & 0 \\ 0 & h_k & h_{k-1} & \cdots & h_0 & \cdots & 0 \\ \vdots & & \ddots & & & \ddots & \vdots \\ 0 & \cdots & 0 & h_k & h_{k-1} & \cdots & h_0 \end{bmatrix}.$$

This matrix satisfies the orthogonality relation

$$GH^T = 0.$$

Why reverse the coefficients?

Since $g(X)h(X) = X^n - 1 \equiv 0 \pmod{X^n - 1}$ and every codeword has the form $c(X) = m(X)g(X)$, we have

$$c(X)h(X) \equiv 0.$$

Writing this condition as inner products of coefficient vectors leads precisely to the reversed-coefficient construction of H .

1.8.1 Example 6.1: Parity-Check Matrix for a $[7, 4]_2$ Cyclic Code

For

$$h(X) = 1 + X + X^2 + X^4,$$

its coefficient vector is $(1, 1, 1, 0, 1)$ and the reversed vector is $(1, 0, 1, 1, 1)$. Therefore

$$H = \begin{bmatrix} 1 & 0 & 1 & 1 & 1 & 0 & 0 \\ 0 & 1 & 0 & 1 & 1 & 1 & 0 \\ 0 & 0 & 1 & 0 & 1 & 1 & 1 \end{bmatrix}.$$

1.8.2 Example 6.2: Verifying Orthogonality

Take the first row of G , $(1, 1, 0, 1, 0, 0, 0)$, and the first row of H , $(1, 0, 1, 1, 1, 0, 0)$. Their inner product is

$$1 \cdot 1 + 1 \cdot 0 + 0 \cdot 1 + 1 \cdot 1 = 1 + 1 = 0 \pmod{2}.$$

Likewise, the first row of G is orthogonal to the second and third rows of H , and similarly for all remaining rows.

1.8.3 Example 6.3: Error Detection with the Parity-Check Matrix

For

$$\mathbf{v} = (1, 1, 1, 1, 1, 1, 1),$$

we get

$$H\mathbf{v}^T = \mathbf{0},$$

so no error is detected.

If instead

$$\mathbf{v}' = (0, 1, 1, 1, 1, 1, 1),$$

then

$$H\mathbf{v}'^T = \begin{bmatrix} 1 \\ 0 \\ 0 \end{bmatrix} \neq \mathbf{0},$$

so the error is detected.

1.9 Systematic and Non-Systematic Encoding

Both methods produce valid codewords, but they place the original message differently inside the codeword.

Category	Non-systematic	Systematic
Encoding rule	$c(X) = m(X)g(X)$	$c(X) = p(X) + X^{n-k}m(X)$
Message position	Mixed throughout the codeword	Preserved explicitly in fixed positions
Ease of decoding	Requires division	Message bits can be read directly
Typical practical use	Rare	Common in real systems

1.9.1 Example 7.1: Two Encodings of the Same Message

Let $g(X) = 1 + X + X^3$ and let the message be $\mathbf{m} = (1, 0, 1, 1)$.

Non-systematic encoding:

$$c(X) = m(X)g(X) = (1 + X^2 + X^3)(1 + X + X^3) = 1 + X + X^2 + X^3 + X^4 + X^5 + X^6.$$

The codeword is $(1, 1, 1, 1, 1, 1, 1)$, and the message is not visibly preserved as a contiguous block.

Systematic encoding: the resulting codeword has the form

$$(1, 0, 0, \mathbf{1}, \mathbf{0}, \mathbf{1}, \mathbf{1}),$$

so the last four bits are exactly the original message.

1.10 Practical Systematic Encoding

Systematic encoding proceeds in three steps.

Step 1. Shift the message polynomial

$$X^{n-k}m(X).$$

Step 2. Compute the remainder

$$p(X) = X^{n-k}m(X) \bmod g(X).$$

Step 3. Form the codeword

$$c(X) = p(X) + X^{n-k}m(X).$$

Thus the codeword has the structure

$$[\text{parity } (n - k) \text{ bits} \mid \text{message } k \text{ bits}].$$

Why is this a valid codeword? Since $p(X)$ is the remainder of $X^{n-k}m(X)$ modulo $g(X)$,

$$X^{n-k}m(X) - p(X)$$

is divisible by $g(X)$. Over $GF(2)$, subtraction equals addition, so $c(X)$ is also divisible by $g(X)$.

1.10.1 Example 8.1: Systematic Encoding of $(1, 0, 1, 1)$

Let $g(X) = 1 + X + X^3$, $n - k = 3$, and $m(X) = 1 + X^2 + X^3$.

Step 1:

$$X^3m(X) = X^3 + X^5 + X^6.$$

Step 2: Divide $X^6 + X^5 + X^3$ by $X^3 + X + 1$.

Step	Dividend	Quotient term
1	$X^6 + X^5 + X^3$ $-(X^6 + X^4 + X^3)$ $= X^5 + X^4$	X^3
2	$X^5 + X^4$ $-(X^5 + X^3 + X^2)$ $= X^4 + X^3 + X^2$	X^2
3	$X^4 + X^3 + X^2$ $-(X^4 + X^2 + X)$ $= X^3 + X$	X
4	$X^3 + X$ $-(X^3 + X + 1)$ $= 1$	1

So the quotient is $X^3 + X^2 + X + 1$, and the remainder is

$$p(X) = 1.$$

Step 3:

$$c(X) = 1 + X^3 + X^5 + X^6.$$

Thus

$$\mathbf{c} = (\underbrace{1, 0, 0}_{\text{parity}}, \underbrace{1, 0, 1, 1}_{\text{message}}).$$

1.10.2 Example 8.2: Systematic Encoding of $(1, 1, 0, 0)$

Here $m(X) = 1 + X$.

Step 1:

$$X^3m(X) = X^3 + X^4.$$

Step 2: Divide by $X^3 + X + 1$ to obtain remainder

$$p(X) = X^2 + 1,$$

which corresponds to parity vector $(1, 0, 1)$.

Step 3:

$$c(X) = 1 + X^2 + X^3 + X^4.$$

So

$$\mathbf{c} = (\underbrace{1, 0, 1}_{\text{parity}}, \underbrace{1, 1, 0, 0}_{\text{message}}).$$

1.10.3 Example 8.3: Systematic Encoding of $(0, 0, 0, 1)$

Here $m(X) = X^3$.

Step 1:

$$X^3m(X) = X^6.$$

Step 2: Dividing by $X^3 + X + 1$ again yields

$$p(X) = X^2 + 1,$$

so the parity bits are $(1, 0, 1)$.

Step 3:

$$c(X) = 1 + X^2 + X^6.$$

Hence

$$\mathbf{c} = (\underbrace{1, 0, 1}_{\text{parity}}, \underbrace{0, 0, 0, 1}_{\text{message}}).$$

1.10.4 Example 8.4: Full Table of Systematic Codewords for the $[7, 4]_2$ Code

Message $(m_0m_1m_2m_3)$	$m(X)$	Parity $(p_0p_1p_2)$	Codeword $(p_0p_1p_2m_0m_1m_2m_3)$
0000	0	000	0000000
1000	1	110	1101000
0100	X	011	0110100
1100	$1 + X$	101	1011100
0010	X^2	111	1110010
1010	$1 + X^2$	001	0011010
0110	$X + X^2$	100	1000110
1110	$1 + X + X^2$	010	0101110
0001	X^3	101	1010001
1001	$1 + X^3$	011	0111001
0101	$X + X^3$	110	1100101
1101	$1 + X + X^3$	000	0001101

Message ($m_0m_1m_2m_3$)	$m(X)$	Parity ($p_0p_1p_2$)	Codeword ($p_0p_1p_2m_0m_1m_2m_3$)
0011	$X^2 + X^3$	010	0100011
1011	$1 + X^2 + X^3$	100	1001011
0111	$X + X^2 + X^3$	001	0010111
1111	$1 + X + X^2 + X^3$	111	1111111

1.11 Syndromes and Error Detection

The **remainder** obtained by dividing the received polynomial $v(X)$ by the generator polynomial $g(X)$ is called the **syndrome**:

$$s(X) = v(X) \bmod g(X).$$

- If $s(X) = 0$, no error is detected.
- If $s(X) \neq 0$, an error is detected.

Core principle: if $v(X) = c(X) + e(X)$ and $c(X)$ is a multiple of $g(X)$, then

$$s(X) = v(X) \bmod g(X) = e(X) \bmod g(X).$$

Thus the syndrome depends only on the error pattern, not on the original message.

1.11.1 Example 9.1: Error-Free Reception

Suppose the codeword

$$c(X) = 1 + X^3 + X^5 + X^6$$

is received without error. Dividing by $X^3 + X + 1$ yields remainder 0.

Step	Dividend	Quotient term
1	$X^6 + X^5 + X^3 + 1$ $-(X^6 + X^4 + X^3)$ $= X^5 + X^4 + 1$	X^3
2	$X^5 + X^4 + 1$ $-(X^5 + X^3 + X^2)$ $= X^4 + X^3 + X^2 + 1$	X^2
3	$X^4 + X^3 + X^2 + 1$ $-(X^4 + X^2 + X)$ $= X^3 + X + 1$	X
4	$X^3 + X + 1$ $-(X^3 + X + 1)$ $= 0$	1

So the syndrome is 0.

1.11.2 Example 9.2: A Single-Bit Error at Position X^2

Suppose the received polynomial is

$$v(X) = 1 + X^2 + X^3 + X^5 + X^6,$$

so the error polynomial is $e(X) = X^2$.

Then

$$s(X) = X^2,$$

whose vector representation is $(0, 0, 1)$. Since the syndrome is nonzero, the error is detected.

1.11.3 Example 9.3: Complete Syndrome Table for Single-Bit Errors in the $[7, 4]_2$ Code

For a single-bit error $e(X) = X^i$, the syndrome is $s(X) = X^i \bmod g(X)$.

Error position i	$e(X)$	$s(X) =$ $e(X) \bmod (1 + X + X^3)$	Syndrome vector
0	1	1	(1, 0, 0)
1	X	X	(0, 1, 0)
2	X^2	X^2	(0, 0, 1)
3	X^3	$1 + X$	(1, 1, 0)
4	X^4	$X + X^2$	(0, 1, 1)
5	X^5	$1 + X + X^2$	(1, 1, 1)
6	X^6	$1 + X^2$	(1, 0, 1)

Since all 7 syndromes are distinct, the code can correct **any single-bit error**.

1.11.4 Example 9.4: Error Correction Using the Syndrome

Suppose the received vector is

$$\mathbf{v} = (1, 0, 1, 1, 0, 1, 1).$$

Step 1. Compute the syndrome: with

$$v(X) = 1 + X^2 + X^3 + X^5 + X^6,$$

we obtain

$$s(X) = X^2,$$

so the syndrome vector is $(0, 0, 1)$.

Step 2. Look up the syndrome table: $(0, 0, 1)$ corresponds to error position $i = 2$.

Step 3. Correct the error:

$$\hat{c}(X) = v(X) + X^2 = 1 + X^3 + X^5 + X^6.$$

Therefore the corrected codeword is

$$\hat{\mathbf{c}} = (1, 0, 0, 1, 0, 1, 1).$$

Step 4. Extract the message: for a systematic code, the last four bits are the message,

$$\mathbf{m} = (1, 0, 1, 1).$$

1.11.5 Example 9.5: The Case of a Two-Bit Error

Suppose the received vector is

$$\mathbf{v} = (0, 1, 0, 1, 0, 1, 1),$$

which corresponds to simultaneous errors at positions 0 and 1. Then

$$e(X) = 1 + X,$$

and the syndrome is

$$s(X) = 1 + X,$$

with vector $(1, 1, 0)$.

But $(1, 1, 0)$ corresponds in the syndrome table to a *single* error at position 3. A decoder designed only for single-error correction will therefore flip the wrong bit, causing a **miscorrection**.

This is exactly the limitation of the $[7, 4]_2$ code: since its minimum distance is $d_{\min} = 3$, it can correct 1 bit or detect 2 bits, but it cannot reliably correct arbitrary 2-bit errors.

1.12 Decoding of Cyclic Codes

The details of decoding depend on the specific family of cyclic code (simple cyclic code, BCH code, RS code, and so on), but the overall structure is usually the same.

1.12.1 Step 1: Syndrome Calculation

Given the received polynomial

$$v(X) = c(X) + e(X),$$

compute

$$s(X) = v(X) \bmod g(X).$$

This can be done very efficiently in hardware by feeding the received stream into an LFSR.

1.12.2 Step 2: Error-Pattern Search

Using the syndrome $s(X)$, determine the actual error polynomial $e(X)$.

- **Syndrome lookup table:** for short block codes (such as small Hamming codes or Golay codes), the syndrome can be mapped directly to an error pattern.
- **Meggitt decoder:** especially hardware-friendly. The syndrome register is shifted one clock at a time while logic checks whether the current pattern indicates an error in the most significant position.
- **Algebraic decoding:** for BCH and Reed-Solomon codes, lookup tables are impractical. One instead computes the error-locator polynomial using methods such as the Berlekamp-Massey algorithm, locates roots by Chien search, and recovers magnitudes via the Forney algorithm.

1.12.3 Step 3: Error Correction

Once an estimated error pattern $\hat{e}(X)$ has been found, subtract it from the received vector:

$$\hat{c}(X) = v(X) - \hat{e}(X).$$

For binary codes, subtraction and addition are the same, so this is just XOR.

1.12.4 Step 4: Message Extraction

Finally, recover the original message.

- **Systematic codes:** simply read the designated message positions in the codeword.
- **Non-systematic codes:** divide the corrected codeword by $g(X)$ and take the quotient.

1.13 How Is the Error-Correcting Capability of a Cyclic Code Determined?

A cyclic code is still a linear code, so its error-correcting capability is determined by its **minimum distance** $d_{\min}$. There is no special correction formula unique to cyclic codes; the same general relationship for linear codes applies:

1.13.1 t -Error-Correcting Capability

A code with minimum distance $d_{\min}$ can correct up to

$$t = \left\lfloor \frac{d_{\min} - 1}{2} \right\rfloor$$

errors.

If one is interested only in error **detection**, then up to $d_{\min} - 1$ errors can be detected.

1.13.2 Example 11.1: Error-Correcting Capability of Representative Cyclic Codes

Code	$[n, k]$	$d_{\min}$	Correction capability t	Detection capability
[7, 4] cyclic code (Hamming)	$[7, 4]_2$	3	1	2
[15, 7] BCH code	$[15, 7]$	5	2	4
[23, 12] Golay code	$[23, 12]$	7	3	6
[15, 5] BCH code	$[15, 5]$	7	3	6
[31, 21] BCH code	$[31, 21]$	5	2	4

1.13.3 Limitation of a General Cyclic Code

For a general cyclic code, there is usually no simple closed-form formula that gives $d_{\min}$ directly from $g(X)$. One must compute it either by enumerating codewords or by a deeper algebraic analysis.

1.13.4 BCH Bound: Guaranteed Designed Distance

A famous theorem does exist for the important subclass of **BCH codes**.

BCH bound: Let α be a primitive n th root of unity satisfying $X^n - 1 = 0$ in $GF(q^m)$. If the generator polynomial $g(X)$ has the consecutive roots

$$\alpha^b, \alpha^{b+1}, \dots, \alpha^{b+\delta-2},$$

then the minimum distance satisfies

$$d_{\min} \geq \delta.$$

The parameter δ is called the **designed distance**. Because of this theorem, one can choose a target correction capability t in advance by setting

$$\delta = 2t + 1.$$

1.13.5 Example 11.2: Applying the BCH Bound

For the binary (15, 7) BCH code, let α be a primitive 15th root of unity in $GF(2^4)$. If the generator polynomial has roots

$$\alpha, \alpha^2, \alpha^3, \alpha^4,$$

then there are 4 consecutive roots, so

$$\delta = 4 + 1 = 5.$$

Therefore

$$d_{\min} \geq 5 \Rightarrow t \geq 2.$$

In fact, this code attains $d_{\min} = 5$, so the BCH bound is tight here.

1.13.6 Example 11.3: General Cyclic Code vs. BCH Code

For the $[7, 4]_2$ cyclic code with

$$g(X) = 1 + X + X^3,$$

let α be a primitive root of $X^7 + 1$ in $GF(2^3)$. Since

$$g(\alpha) = 0, \quad g(\alpha^2) = 0,$$

there are two consecutive roots. Hence the BCH bound gives

$$\delta = 3 \Rightarrow d_{\min} \geq 3.$$

The actual minimum distance is exactly 3. In fact this code is both the $[7, 4]_2$ Hamming code and a BCH code.

This ability to design error-correcting performance at the construction stage is one of the main reasons BCH and Reed-Solomon codes are favored in practice over arbitrary cyclic codes.

1.14 Algebraic Structure of Cyclic Codes

A cyclic code is a **principal ideal** of the quotient ring

$$R_n = \mathbb{F}_q[X]/\langle X^n - 1 \rangle.$$

This means two things:

1. The cyclic code $\mathcal{C}$ is an ideal of the ring R_n .
2. This ideal is generated by a single element $g(X)$:

$$\mathcal{C} = \langle g(X) \rangle.$$

1.14.1 Proof

Step 1: Show that $\mathcal{C}$ is an ideal.

- **Closed under addition:** by linearity, if $a(X), b(X) \in \mathcal{C}$ then $a(X) + b(X) \in \mathcal{C}$.

- **Absorption:** for any $r(X) \in R_n$ and $c(X) \in \mathcal{C}$,

$$r(X)c(X) \in \mathcal{C}.$$

This follows because cyclic-shift invariance implies $Xc(X) \in \mathcal{C}$, repeated shifts give $X^2c(X), X^3c(X), \dots$, and linearity then gives closure under any polynomial multiple.

Step 2: Show that the ideal is principal.

- Let $g(X)$ be the monic polynomial of smallest degree in $\mathcal{C}$.
- For any $c(X) \in \mathcal{C}$, divide:

$$c(X) = q(X)g(X) + r(X), \quad \deg r < \deg g.$$

- Since $r(X) = c(X) - q(X)g(X) \in \mathcal{C}$ and $g(X)$ has minimal degree, we must have $r(X) = 0$.
- Hence every codeword is a multiple of $g(X)$, so

$$\mathcal{C} = \langle g(X) \rangle.$$

1.14.2 Example 10.1: Checking the Ideal Structure

For the $[7, 4]_2$ code with $g(X) = 1 + X + X^3$:

- $g(X) \in \mathcal{C}$
- $Xg(X) = X + X^2 + X^4 \in \mathcal{C}$
- $(1 + X)g(X) = 1 + X^2 + X^3 + X^4 \in \mathcal{C}$

This illustrates both cyclic closure and absorption.

1.14.3 Example 10.2: Why Must $g(X)$ Divide $X^n - 1$?

For $g(X) = 1 + X + X^3$,

$$X^7 + 1 = (1 + X + X^3)(1 + X + X^2 + X^4) = g(X)h(X).$$

Thus $g(X)$ divides $X^7 + 1$, which is a necessary condition for any generator polynomial.

1.15 Number of Ideals in the Quotient Ring and Determination of the Generator Polynomial

From the previous section, cyclic codes correspond one-to-one with principal ideals of the quotient ring

$$R_n = \mathbb{F}_q[X]/\langle X^n - 1 \rangle.$$

This naturally leads to two questions:

1. How many ideals are there?
2. How is the generator polynomial that defines each ideal determined?

1.15.1 Main Theorem: Counting Ideals

Every ideal of R_n is a principal ideal generated by a monic divisor $g(X)$ of $X^n - 1$:

$$I = \langle g(X) \rangle.$$

Proof.

Consider the canonical projection

$$\pi : \mathbb{F}_q[X] \rightarrow R_n, \quad f(X) \mapsto f(X) + \langle X^n - 1 \rangle.$$

This is a surjective ring homomorphism with kernel $\langle X^n - 1 \rangle$.

By the correspondence theorem for rings, there is a one-to-one correspondence between ideals I of R_n and ideals J of $\mathbb{F}_q[X]$ satisfying

$$\langle X^n - 1 \rangle \subseteq J.$$

Specifically,

$$I = J / \langle X^n - 1 \rangle, \quad J = \pi^{-1}(I).$$

Since $\mathbb{F}_q[X]$ is a PID (indeed a Euclidean domain), every ideal J is principal:

$$J = \langle g(X) \rangle$$

for a unique monic polynomial $g(X)$.

The inclusion $\langle X^n - 1 \rangle \subseteq \langle g(X) \rangle$ is equivalent to

$$g(X) \mid (X^n - 1).$$

Conversely, every monic divisor $g(X)$ of $X^n - 1$ generates an ideal of R_n .

Therefore there is a one-to-one correspondence:

$$g(X) \mid (X^n - 1) \iff \langle g(X) \rangle \text{ is an ideal of } R_n.$$

■

Hence:

$$\text{Number of ideals of } R_n = \text{number of monic divisors of } X^n - 1 \text{ over } \mathbb{F}_q[X].$$

If $\gcd(n, q) = 1$, so that $X^n - 1$ has no repeated roots, and if

$$X^n - 1 = f_1(X)f_2(X) \cdots f_s(X)$$

is a product of s distinct irreducible polynomials, then each factor may be independently included or excluded from a divisor. Therefore the total number of monic divisors is

$$2^s.$$

This count includes both the whole ring ($g(X) = 1$) and the zero ideal ($g(X) = X^n - 1$).

Why 2^s ? Every monic divisor corresponds uniquely to a subset $S \subseteq \{1, 2, \dots, s\}$ through

$$g_S(X) = \prod_{i \in S} f_i(X).$$

Since the number of subsets is 2^s , the number of monic divisors is also 2^s .

1.15.2 Procedure for Determining Generator Polynomials

Step 1. Factor $X^n - 1$ completely into irreducible polynomials over $\mathbb{F}_q[X]$.

Step 2. Take products over all subsets of those irreducible factors.

Step 3. For each generator polynomial $g(X)$, the corresponding cyclic code has parameters

$$(n, n - \deg g(X)).$$

1.15.3 Example 13.1: Listing All Ideals for $n = 7$ over $\mathbb{F}_2$

Since

$$X^7 + 1 = (1 + X)(1 + X + X^3)(1 + X^2 + X^3),$$

there are $s = 3$ distinct irreducible factors, so the number of ideals is

$$2^3 = 8.$$

Subset S	$g(X) = \prod_{i \in S} f_i(X)$	$\deg(g)$	$[n, k]$	Number of codewords 2^k	Remark
$\emptyset$	1	0	$(7, 7)$	128	Whole ring (trivial)
$\{1\}$	$1 + X$	1	$(7, 6)$	64	Simple parity-check code
$\{2\}$	$1 + X + X^3$	3	$[7, 4]_2$	16	Hamming $(7, 4)$
$\{3\}$	$1 + X^2 + X^3$	3	$[7, 4]_2$	16	
$\{1, 2\}$	$1 + X^2 + X^3 + X^4$	4	$(7, 3)$	8	Repetition code
$\{1, 3\}$	$1 + X + X^2 + X^4$	4	$(7, 3)$	8	
$\{2, 3\}$	$1 + X + X^2 + X^3 + X^4 + X^5 + X^6$	6	$(7, 1)$	2	
$\{1, 2, 3\}$	$X^7 + 1$	7	$(7, 0)$	1	Zero ideal (trivial)

These 8 ideals form a lattice under inclusion: if

$$g_1(X) \mid g_2(X),$$

then

$$\langle g_2(X) \rangle \subseteq \langle g_1(X) \rangle.$$

So larger generator-polynomial degree means smaller code dimension and a smaller ideal.

1.15.4 Example 13.2: Number of Ideals for $n = 15$ over $\mathbb{F}_2$

From Exercise 2,

$$X^{15} + 1 = (1 + X)(1 + X + X^2)(1 + X + X^4)(1 + X^3 + X^4)(1 + X + X^2 + X^3 + X^4).$$

There are $s = 5$ irreducible factors, so the number of ideals is

$$2^5 = 32.$$

Among these 32 ideals, there are already several $(15, 11)$ cyclic codes, depending on which degree-4 factor or combination of factors is selected.

1.15.5 Example 13.3: Ideal Lattice and Inclusion Relations

For $n = 7$, the inclusion lattice looks like this:

```

      <1> = R7  (whole ring, k=7)
    /  |  \
  <f1> <f2> <f3>   (k=6,4,4)
  | \ / \ / |
<f1f2> <f1f3> <f2f3>   (k=3,3,1)
  \  |  /
  <f1f2f3> = <0>   (zero ideal, k=0)

```

As you move upward, the ideals get larger and contain more codewords. As you move downward, the ideals get smaller, the code dimension decreases, and the error-correcting capability generally improves.

1.15.6 A Caution Concerning $\gcd(n, q) \neq 1$

If $\gcd(n, q) \neq 1$, then $X^n - 1$ has repeated roots over $\mathbb{F}_q[X]$. For example, over $\mathbb{F}_2$,

$$X^6 + 1 = (1 + X)^2(1 + X + X^2)^2,$$

since $\gcd(6, 2) = 2$.

In that case, one may independently choose exponents

$$0 \leq e'_i \leq e_i$$

for each repeated factor $f_i(X)^{e_i}$, and the number of monic divisors becomes

$$\prod_{i=1}^s (e_i + 1).$$

For the example above, that is

$$(2 + 1)(2 + 1) = 9,$$

which is larger than $2^s = 4$.

1.16 Why Are Cyclic Codes Important in Practice?

The main practical strength of cyclic codes is that they admit **ultra-fast real-time hardware implementations** using LFSRs.

1.16.1 The Principle of the LFSR Structure

Whereas a general linear block code requires large matrix multiplications, a cyclic code reduces to lightweight polynomial division implemented by:

- $r = \deg g(X)$ flip-flops,
- a few XOR gates matching the nonzero coefficients of the polynomial,
- and nothing more.

1.16.2 Advantage of Real-Time Processing

General linear code	Cyclic code (LFSR)
Buffer the whole block, then multiply by a matrix	Bit-by-bit real-time processing
Large memory required	Only r registers required
High latency	Near-zero latency
Complex parallel arithmetic	Simple serial XOR circuit

1.16.3 CRC in Network Transmission

When checking the integrity of gigabit data streams in USB or Ethernet:

1. **No large memory is needed:** data can be processed as it arrives.
2. **On-the-fly computation:** one bit per clock is fed into the LFSR.
3. **Immediate result:** when the last bit passes, the remaining register contents are the parity bits or syndrome.

1.16.4 Example 11.1: LFSR Encoder for $g(X) = 1 + X + X^3$

For systematic encoding with $g(X) = 1 + X + X^3$, use three flip-flops $[R_0, R_1, R_2]$. The feedback is the XOR of the incoming bit and the most significant register R_2 . Since the coefficients of X^0 and X^1 are 1, the feedback is XORed into R_0 and R_1 .

Feed the message bits $(1, 0, 1, 1)$ in order from highest degree to lowest, i.e. m_3, m_2, m_1, m_0 :

Clock	Input bit	Feedback = input $\oplus R_2$	R_0	R_1	R_2
Initial	-	-	0	0	0
1	$m_3 = 1$	$1 \oplus 0 = 1$	1	0	0
2	$m_2 = 1$	$1 \oplus 0 = 1$	1	1	0
3	$m_1 = 0$	$0 \oplus 0 = 0$	0	1	1
4	$m_0 = 1$	$1 \oplus 1 = 0$	0	0	1

After 4 clocks the register state is $(R_0, R_1, R_2) = (0, 0, 1)$.

Verification: In Example 8.1, the remainder for systematic encoding of $m(X) = 1 + X^2 + X^3$ was $p(X) = 1$, i.e. parity vector $(1, 0, 0)$. Depending on whether the circuit is interpreted MSB-first or LSB-first, the register output order may be reversed. Reading $(0, 0, 1)$ backward gives $(1, 0, 0)$.

Remark: The exact behavior of an LFSR depends on the feedback topology and the bit-input convention. The essential point is that polynomial division can be executed in real time using only $n - k$ registers and a few XOR gates.

1.17 Representative Families of Cyclic Codes

Important code families with cyclic structure include:

Code	Characteristic	Main applications
CRC (Cyclic Redundancy Check)	Error detection only, many standard generator polynomials	Ethernet, USB, ZIP, storage media
BCH codes	Root-based construction guaranteeing error-correction capability	Flash memory, satellite communication
Reed-Solomon (RS) codes	Nonbinary cyclic code, symbol-level error correction	CD/DVD/Blu-ray, QR codes, deep-space communication
Some Hamming codes	Cyclic form for certain parameters	ECC memory

1.17.1 Example 15.1: CRC-8 Generator Polynomial

A standard CRC-8 polynomial is

$$g(X) = X^8 + X^2 + X + 1.$$

- Code length: variable (depends on the message length)
- Parity length: 8 bits
- Typical use: sensor networks and embedded systems

1.17.2 Example 15.2: CRC-32 Used in Ethernet

$$g(X) = X^{32} + X^{26} + X^{23} + X^{22} + X^{16} + X^{12} + X^{11} + X^{10} + X^8 + X^7 + X^5 + X^4 + X^2 + X + 1.$$

- 32 parity bits
- Protects frames up to length $2^{32} - 1$ in theory
- Used for the FCS (Frame Check Sequence) in Ethernet

1.18 What Are Quasi-Cyclic Codes and the Post-Quantum Cryptosystem HQC?

The algebraic structure of cyclic codes plays an important role in modern cryptography, especially in **post-quantum cryptography (PQC)**.

1.18.1 Quasi-Cyclic Codes (QC)

If a cyclic code is closed under a 1-step shift, then a **quasi-cyclic code** is closed under an ℓ -step shift.

- If the code length is $n = m\ell$, then shifting any codeword by ℓ positions preserves membership in the code.
- If $\ell = 1$, we recover an ordinary cyclic code.
- Generator and parity-check matrices are built from $m \times m$ **circulant blocks**.

1.18.2 Use in HQC (Hamming Quasi-Cyclic)

HQC is a code-based key encapsulation mechanism designed to remain secure against quantum computers, including attacks based on Shor's algorithm. It uses quasi-cyclic structure in a central way.

The classical problem: the McEliece cryptosystem is highly secure but suffers from very large public keys, typically around the megabyte scale.

The HQC idea: adopt a quasi-cyclic structure in order to obtain:

1. **Smaller public keys:** a circulant matrix is determined entirely by its first row, so one polynomial can represent a full matrix block.
2. **Faster arithmetic:** operations are performed in a polynomial ring such as $\mathbb{F}_2[X]/\langle X^n - 1 \rangle$, enabling efficient algorithms.
3. **Provable security reductions:** the scheme is related to hard syndrome-decoding problems believed to remain difficult even for quantum computers.

1.18.3 Example 16.1: Structure of a Circulant Block

Consider the 4×4 circulant matrix whose first row is $(1, 0, 1, 1)$:

$$C = \begin{bmatrix} 1 & 0 & 1 & 1 \\ 1 & 1 & 0 & 1 \\ 1 & 1 & 1 & 0 \\ 0 & 1 & 1 & 1 \end{bmatrix}.$$

Each row is obtained from the previous one by a one-step cyclic right shift. Therefore storing only the first row is enough to reconstruct the entire matrix. This is precisely why quasi-cyclic structure can reduce public-key size so dramatically.

1.19 Exercises

1.19.1 Exercise 1. Determining Whether a Code Is Cyclic

Decide whether the following code is cyclic. ($n = 5$)

$$\mathcal{C} = \{00000, 10110, 01011, 11101, 01101, 10011, 00111, 11010, 11001, 00101, \dots\}$$

Hint: apply a one-step cyclic shift to each codeword and check whether the result is still in the set.

1.19.2 Exercise 2. Finding Generator Polynomials

Suppose that over $GF(2)$,

$$X^{15} + 1 = (1 + X)(1 + X + X^2)(1 + X + X^4)(1 + X^3 + X^4)(1 + X + X^2 + X^3 + X^4).$$

- (a) List all degree-4 generator polynomials that can define a $(15, 11)$ cyclic code.
 - (b) If $g(X) = 1 + X + X^4$, compute the corresponding parity-check polynomial $h(X)$.
-

1.19.3 Exercise 3. Practice with Systematic Encoding

For the $[7, 4]_2$ cyclic code with $g(X) = 1 + X + X^3$, systematically encode the following messages:

- (a) $\mathbf{m} = (0, 1, 1, 0)$
 - (b) $\mathbf{m} = (1, 1, 1, 1)$
 - (c) Check that your results agree with the codeword table in Example 8.4.
-

1.19.4 Exercise 4. Syndrome Computation and Error Correction

For the $[7, 4]_2$ cyclic code, compute the syndrome of each received vector and correct it if it contains a single-bit error:

- (a) $\mathbf{v} = (1, 1, 0, 1, 0, 0, 0)$
- (b) $\mathbf{v} = (0, 1, 1, 0, 1, 0, 1)$
- (c) $\mathbf{v} = (1, 0, 1, 1, 1, 0, 0)$

Hint: use the syndrome table of Example 9.3.

1.19.5 Exercise 5. Generator and Parity-Check Matrices

Consider the $[7, 4]_2$ cyclic code with generator polynomial

$$g(X) = 1 + X^2 + X^3.$$

- (a) Find the non-systematic generator matrix G .
 - (b) Compute the parity-check polynomial $h(X)$.
 - (c) Construct the parity-check matrix H .
 - (d) Verify that $GH^T = 0$.
-

1.19.6 Exercise 6. Understanding the Algebraic Structure

- (a) Factor $X^6 + 1$ over $GF(2)$.

Caution: since $\gcd(6, 2) = 2 \neq 1$, the polynomial $X^6 + 1$ has repeated roots over $GF(2)$, so some standard results of cyclic-code theory do not apply in their usual form. Confirm from the factorization that repeated factors occur.

- (b) Based on that factorization, list the divisors of $X^6 + 1$ and determine the corresponding $[n, k]$ parameters.
 - (c) Find the generator polynomial of the code with largest dimension, and determine its minimum distance.
-

1.19.7 Exercise 7. Comprehensive Problem

Let the generator polynomial of a $(15, 7)$ BCH code be

$$g(X) = 1 + X^4 + X^6 + X^7 + X^8.$$

- (a) Verify that this is a cyclic code by checking that $g(X)$ divides $X^{15} + 1$.
 - (b) Systematically encode the message
 $\mathbf{m} = (1, 0, 0, 0, 0, 0, 1)$.
 - (c) Verify that the syndrome of the encoded codeword is 0.
-

1.20 SageMath Code

Running the following SageMath script allows you to verify the main concepts discussed in this document directly.

1.21 References

- [1] C. E. Shannon, "A Mathematical Theory of Communication," *Bell System Technical Journal*, vol. 27, no. 3, pp. 379-423, 1948.
- [2] R. W. Hamming, "Error Detecting and Error Correcting Codes," *Bell System Technical Journal*, vol. 29, no. 2, pp. 147-160, 1950.
- [3] D. E. Muller, "Application of Boolean Algebra to Switching Circuit Design and to Error Detection," *IRE Transactions on Electronic Computers*, vol. EC-3, no. 3, pp. 6-12, 1954.

- [4] I. S. Reed, "A Class of Multiple-Error-Correcting Codes and the Decoding Scheme," *IRE Transactions on Information Theory*, vol. 4, no. 4, pp. 38-49, 1954.
- [5] E. Prange, "Cyclic Error-Correcting Codes in Two Symbols," *AFCRC-TN-57-103*, Air Force Cambridge Research Center, 1957.
- [6] A. Hocquenghem, "Codes correcteurs d'erreurs," *Chiffres*, vol. 2, pp. 147-156, 1959.
- [7] R. C. Bose and D. K. Ray-Chaudhuri, "On a Class of Error Correcting Binary Group Codes," *Information and Control*, vol. 3, no. 1, pp. 68-79, 1960.
- [8] I. S. Reed and G. Solomon, "Polynomial Codes Over Certain Finite Fields," *Journal of the Society for Industrial and Applied Mathematics*, vol. 8, no. 2, pp. 300-304, 1960.
- [9] W. W. Peterson, *Error-Correcting Codes*, MIT Press, 1961.
- [10] E. R. Berlekamp, *Algebraic Coding Theory*, McGraw-Hill, 1968.
- [11] J. L. Massey, "Shift-Register Synthesis and BCH Decoding," *IEEE Transactions on Information Theory*, vol. 15, no. 1, pp. 122-127, 1969.
- [12] V. D. Goppa, "Geometry and Codes," *Mathematics and its Applications*, vol. 24, Kluwer, 1988.
- [13] R. J. McEliece, "A Public-Key Cryptosystem Based on Algebraic Coding Theory," *DSN Progress Report*, Jet Propulsion Laboratory, 1978.
- [14] C. Berrou, A. Glavieux, and P. Thitimajshima, "Near Shannon Limit Error-Correcting Coding and Decoding: Turbo Codes," in *Proceedings of ICC '93*, 1993.
- [15] D. J. C. MacKay and R. M. Neal, "Near Shannon Limit Performance of Low Density Parity Check Codes," *Electronics Letters*, vol. 32, no. 18, pp. 1645-1646, 1996.